

NSBM Green University

BSc. Honours in Computer Networks

KERNEL-LEVEL DDOS DETECTION USING eBPF XDP

By Shanirangala Ashani Gamage

Supervised By Mr. Chamara Disanayake

Acknowledgement

I would like to begin by thanking my supervisor, Mr. Chamara Disanayake, for his guidance, patience, and honest feedback throughout this project. Working with a technology as unforgiving as eBPF meant that a great deal of this dissertation was learned by getting things wrong first, and his willingness to keep pointing me back in the right direction, rather than simply toward the easiest path, shaped both the system and the way I have written about it.

I am grateful to NSBM Green University and the Department of Computer Networks for the resources, the environment, and the two months in which this work had to be planned, built, and evaluated. I would also like to thank the researchers whose published work is reviewed in Chapter Two - their willingness to report limitations honestly, not only results, made it far easier to see where a real gap in the literature actually was.

Finally, I want to thank my family and friends, who put up with a great deal of talk about kernels, packets, and thresholds over the past two months, and who kept me steady on the days the eBPF verifier did not. This dissertation would not exist without their patience and support.

Abstract

Distributed denial-of-service attacks continue to grow in both scale and frequency, and defences that filter traffic inside the Linux kernel using eBPF and XDP have become an

attractive way to stop them at the earliest possible point in the network stack. These kernel-resident filters are extremely fast, but the very constraints that make them safe - a strict instruction-count limit, no unbounded loops, no floating-point arithmetic - also make them coarse: most rely on a single, fixed packet-rate threshold, which cannot tell a real attack apart from a legitimate flash crowd. Machine-learning classifiers discriminate far better, but are usually applied to all traffic in userspace, which is too slow to sit on the packet path, and the great majority of published work measures only how well a system detects attacks, never how often it wrongly blocks legitimate users during a surge.

This dissertation presents Cerberus, a two-tier, online-adaptive DDoS mitigation prototype that addresses this gap head-on. A lightweight XDP program attached at the network driver counts packets per source IP address and compares that count against a threshold held in a BPF map, dropping or passing each packet with a single bounded operation. Because the threshold lives in a shared map rather than being compiled into the program, a userspace control loop can rewrite it at any time from recent traffic statistics, and the very next packet is judged against the new value with no reload of the kernel program. Sources whose counts fall within an ambiguous band are escalated selectively to a decision-tree classifier trained offline on the CIC-IDS-2017 dataset, keeping the more expensive classification step off the fast path entirely.

The prototype was implemented and evaluated on a live Ubuntu host. The classifier reached an overall accuracy of 0.86 when trained and tested against 702,718 real, labelled samples. In a controlled comparison, a static-threshold baseline and the adaptive two-tier design both detected a volumetric attacker in full, but the static baseline wrongly blocked all twelve simulated legitimate users during a flash-crowd scenario, while the adaptive design blocked none of them. This result supports the central claim of the dissertation: that a two-tier, adaptively-thresholded design can preserve attack detection while substantially reducing the harm a DDoS defence causes to real users during a legitimate traffic surge, addressing a dimension of the problem that existing eBPF/XDP literature does not evaluate.

Contents

CHAPTER 1 - INTRODUCTION

1.1 Chapter Overview

1.2 Problem Background - What Has Actually Happened?

1.3 Problem Statement

1.3.1 General Problem

1.3.2 Specific Problem

1.4 Research Question

1.5 Research Motivation

1.6 Research Aim

1.7 Research Objective

1.7.1 To identify

1.7.2 To analyse

1.7.3 To design and develop

1.7.4 To evaluate

1.8 Rich Picture of the Proposed Solution

1.9 Resource Requirements

1.9.1 Hardware

1.9.2 Software

1.10 Project Scope

1.11 Chapter Summary

CHAPTER 2 - LITERATURE REVIEW

2.1 Chapter Overview

2.2 Conceptual Map of the Literature

2.3 Domain Overview

2.4 Existing Systems, Frameworks and Designs

2.5 Technological Analysis

2.5.1 Algorithmic Analysis

2.5.2 Design Analysis

2.6 Reflection

CHAPTER 3 - METHODOLOGY

- 3.1 Research Paradigm
- 3.2 Research Approach
- 3.3 Research Strategy
- 3.4 Fact Collection Mechanisms
- 3.5 Research Methodology Execution Workflow
- 3.6 Project Management Methodology
 - 3.6.1 Project Timeline
 - 3.6.2 Ethical Considerations
- 3.7 Chapter Summary

CHAPTER 4 - SYSTEM REQUIREMENT SPECIFICATION

- 4.1 Chapter Overview
- 4.2 Stakeholder Analysis
- 4.3 Operationalisation Process
- 4.4 System / Model Analysis
 - 4.4.1 Use Case Diagram
 - 4.4.2 Class Diagram
 - 4.4.3 Activity Diagram
 - 4.4.4 Sequence Diagrams
 - 4.4.5 Deployment Diagram
- 4.5 Proposed System Architecture
- 4.6 Functional and Non-functional Requirements
- 4.7 Chapter Summary

CHAPTER 5 - IMPLEMENTATION / DESIGNING

- 5.1 Building the Framework
 - 5.1.1 Algorithm and Logic
 - 5.1.3 Justification of Technologies
- 5.2 Significant Implementation Attempts

5.3 Chapter Summary

CHAPTER 6 - TESTING AND EVALUATION

6.1 Chapter Overview

6.2 Test Plan and Test Cases

6.2.1 Functional Testing

6.2.2 Non-functional Testing

6.3 Testing / Evaluation Workflow

6.4 Results

6.5 Honest Limitations of the Evaluation

6.6 Review of Test Strategies Used

6.7 Chapter Summary

CHAPTER 7 - CONCLUDING REMARKS

7.1 Accomplishment of the Research Objectives

7.2 Problems Encountered

7.3 Self-reflection

7.3.1 Ideology About the Research Carried Out

7.3.2 Benefits Gained

7.3.3 Learning Curves

7.4 Business Insight of Your Idea

7.4.1 Real-world Application Possibilities

7.5 Future Recommendations

References

Appendix

Appendix A: Tier-1 Kernel Source Listing

Appendix B: Additional Test Cases

Appendix C: Additional Use Case Specifications

CHAPTER 1 - INTRODUCTION

1.1 Chapter Overview

This chapter sets out the research carried out on kernel-level DDoS detection using eBPF and XDP. It gives the background that motivates the study, states the problem in general and specific terms, and identifies the research question along with the motivation, aim, and objectives that guide the rest of the work. The chapter also sketches the proposed solution, lists the resources it needs, and marks out the boundary of what the project does and does not attempt.

1.2 Problem Background - What Has Actually Happened?

Denial of Service attacks are not new, but they have grown sharply in scale, frequency, and sophistication over the past two years. According to Cloudflare's threat intelligence report, in 2025 they blocked 47.1 million DDoS attacks on their network, up from 22.1 million in 2024, and the first quarter of 2025 alone accounted for 96 percent of all DDoS attacks blocked throughout the whole of 2024. A single attack, peaking at 31.4 terabits per second and 14.1 billion packets per second, lasted just 35 seconds before being automatically mitigated, as part of an internal campaign dubbed "The Night Before Christmas" that occurred in December 2025. The botnet behind it, dubbed Aisuru-Kimwolf, is believed to consist of anywhere from one to four million "smart" Internet of Things devices, most of them Android-based TVs, and it can deliver simultaneous packet-rate and bandwidth attacks across both TCP and UDP.

These numbers matter to this dissertation in two ways. First, the sheer volume of present-day attacks makes it clear that detection at the application layer, or even within the standard kernel network stack, is structurally unable to keep up - by the time an attack reaches a userspace process, or even a fully-formed kernel socket buffer, the system may already be overloaded. Second, the growing share of hyper-volumetric and protocol-level floods, together with the persistence of application-layer floods that mimic trusted users, makes a static rule unreliable: traffic from a popular promotional campaign can look almost identical to traffic from a flood attack in one month, and quite different from it the next. It is this second observation - the difficulty of telling a real surge apart from a malicious one, and doing so within microseconds - that shapes the specific research gap this chapter builds towards.

1.3 Problem Statement

Network operators today face a real dilemma. The sooner a bad packet can be dropped, the less CPU time an attacker is able to steal from the system, because the packet never reaches the layers of the operating system that would otherwise have to process it. The eBPF/XDP subsystem of the Linux kernel is a strong answer to this half of the problem: verified, sandboxed programs are attached at the network driver hook itself, so packets can be classified and dropped before the kernel even allocates a full socket buffer structure for them. At the same time, the very features that make eBPF/XDP safe and fast - a small maximum instruction count, a limited per-program stack, and no unbounded loops - make it hard to place any real intelligence inside that hook. Published systems tend to fall into two camps as a result, and both camps share a shortcoming this dissertation tries to correct.

1.3.1 General Problem

In essence, this paradox forces organisations to choose between speed and intelligence. Static, threshold-based filters compiled into XDP - for example, dropping any source IP whose packet rate crosses a fixed number - are cheap enough to run on every packet, but cannot tell a real traffic surge (a flash sale, a viral post, a major sporting result) from an attack, since both produce the same sudden spike against the same fixed number. On the other hand, anything that needs the model itself to change, which in turn requires an update to the eBPF program, is disruptive to carry out while the system is actively defending live traffic; decision trees and quantised neural networks can be trained to recognise far more nuanced attack patterns, but only offline, before the program is loaded. Either way, the defence ends up acting too simply or too slowly for the pace of real traffic and real attackers.

1.3.2 Specific Problem

At the level of this specific research domain, the shortfall can be stated more precisely. Three trends recur across the recent eBPF/XDP DDoS literature. The mitigation results reported for static, rate-based systems tested against synthetic flood traffic such as hping3 look good against a flood, but none of the studies reviewed test the detection logic against a burst of traffic that is deliberately legitimate, so the chance that a real user gets blocked is never actually measured. Some machine-learning systems compiled around eBPF - decision-tree and quantised neural-network classifiers among them - report strong accuracy figures, mostly

above 95 percent, but these figures come from labelled attack datasets such as CIC-IDS-2017 and CICDDoS2019, and the models that are actually deployed stay static once trained. The single closest piece of work, a 2025 framework that builds adaptive, dynamically-thresholded detection logic straight into OpenFlow switches, does solve the adaptivity problem, but only by recalculating a statistical threshold rather than adjusting a learned model, and it depends architecturally on an SDN controller, which limits how it could be used in a host-level, controller-independent deployment.

None of these features is individually well supported by published work, and the combination of all three appears nowhere: (a) a lightweight, two-tier detection pipeline in which only ambiguous traffic is escalated from a cheap statistical pre-filter to a small machine-learning classifier; (b) an online mechanism by which a userspace control loop can adjust the escalation threshold that governs when traffic reaches that classifier, without recompiling or reloading the XDP program; and (c) an evaluation methodology that explicitly measures false positives against legitimate flash-crowd traffic, rather than against attack traffic alone. The gap this dissertation targets in the research literature is that combination.

1.4 Research Question

How can a kernel-level eBPF/XDP DDoS detection system dynamically adjust its detection threshold without reloading the running program, while preserving the accuracy of attack detection and significantly lowering false positives when legitimate network traffic suddenly increases?

This primary question is answered in several ways across the dissertation, and it splits into two sub-questions: what architecture is needed to let a userspace control process safely and efficiently update kernel-resident detection parameters without simply moving the problem back up into userspace, and what evaluation methodology is needed to show, with evidence, that adaptivity actually changes the trade-off between attack detection and false-positive rate, rather than just shifting where that trade-off is made.

1.5 Research Motivation

This research is not only practical, it is personal as well. From a practical point of view, the statistics cited in Section 1.2 make it clear that DDoS has stopped being an occasional event

and has become, in the words of Zayo's 2026 Cybersecurity Insights Report, "a permanent and highly disruptive reality for all organisations." Akamai and Cloudflare are enterprise-grade scrubbing providers, but small and medium-sized organisations face the same hyper-volumetric and adaptive attacks that large organisations are working to combat, usually without the budget for that kind of protection. A lightweight, self-tuning, host-level defence that can be deployed on an ordinary Linux server, without needing a Software-Defined Networking controller or a dedicated scrubbing centre, therefore has direct practical value.

On a more personal level, the appeal of the topic lay in the chance to work at the actual interface between operating-systems research and applied security. What started as a fairly broad idea - "DDoS detection using eBPF/XDP" - and the wish to work inside the Linux kernel while engaging with a live and growing security problem, is what narrowed down into the specific research gap identified above.

1.6 Research Aim

This work aims to design, implement, and empirically assess a two-tier, online-adaptive DDoS detection and mitigation solution built on eBPF/XDP, one that suppresses false positives on legitimate flash-crowd traffic while maintaining detection accuracy against representative DDoS attack traffic, and to show through empirical evaluation that this solution outperforms existing static-threshold baselines.

1.7 Research Objective

1.7.1 To identify

To identify the limitations of existing eBPF/XDP approaches to DDoS detection, and among these, the issues around adaptivity and the way these systems are tested against legitimate traffic surges.

1.7.2 To analyse

To compare a statistical threshold approach against a compiled machine-learning approach under the constraints imposed by the eBPF verifier, in order to design a hybrid two-tier architecture.

1.7.3 To design and develop

To build a working prototype that combines a lightweight machine-learning classifier, invoked selectively rather than on every packet, with a low-cost, kernel-resident statistical pre-filter at the XDP hook, together with a userspace control-loop daemon that can change the detection threshold via a BPF map without reloading the XDP program.

1.7.4 To evaluate

To experimentally test the prototype against both synthetic DDoS attack traffic and deliberately generated legitimate flash-crowd traffic, measuring detection accuracy, false-positive rate, throughput, and CPU overhead, and to compare the results against a static-threshold baseline.

1.8 Rich Picture of the Proposed Solution

The proposed architecture can be thought of as two co-operating parts that sit either side of the kernel/userspace boundary. The XDP program is attached to the network interface driver and receives every packet before the kernel builds any further structures for it. For each IPv4 packet, it performs one cheap, bounded operation: it increments a per-source-IP counter held in a BPF hash map and compares that count against a threshold held in a separate BPF array map, rather than one hard-coded into the program. If the count has reached the threshold, the packet is dropped immediately with `XDP_DROP`; otherwise it is passed on with `XDP_PASS`. This part of the design is deliberately minimal - a lookup, an increment, and a comparison - so that it stays well inside the instruction and complexity limits the eBPF verifier imposes.

The second part is a userspace daemon that manages the pipeline alongside the kernel. It periodically reads the aggregate statistics out of the BPF maps, works out a new baseline for "normal" traffic using an exponentially weighted moving average, and writes an updated threshold back into the BPF map that the kernel program reads on the very next packet. Because this value can be changed from userspace at any time without unloading the program, the detection logic that applies at three in the morning, when traffic is quiet, can be different from the logic in place during a lunchtime sale, and the transition between the two happens automatically.

Sources whose counts sit in an ambiguous band - busy enough to be suspicious, but not yet

over the hard threshold - are escalated to a small decision-tree classifier. Unlike the statistical pre-filter, this classifier is not compiled into the XDP program itself; it runs in userspace and is invoked only for that ambiguous minority of sources, so the more expensive classification step never sits on the packet path. This mirrors a design choice also made by comparable published systems - the SmartX Intelligent Sec framework, for example, keeps its BiLSTM-based classifier at the user level for the same reason, describing the trade-off as a limitation to be revisited in future work rather than a flaw that undermines the contribution. The classifier is trained offline, and its output feeds back into the same userspace control loop that manages the threshold, so that a consistently-suspicious source can influence how quickly the threshold reacts.

A separate evaluation harness, conceptually apart from the running defence but central to the research contribution, replays both attack traffic - generated with hping3 - and synthetic flash-crowd traffic that mimics a real burst of legitimate connections from many distinct sources, and logs the resulting detection accuracy and false-positive behaviour for each scenario.

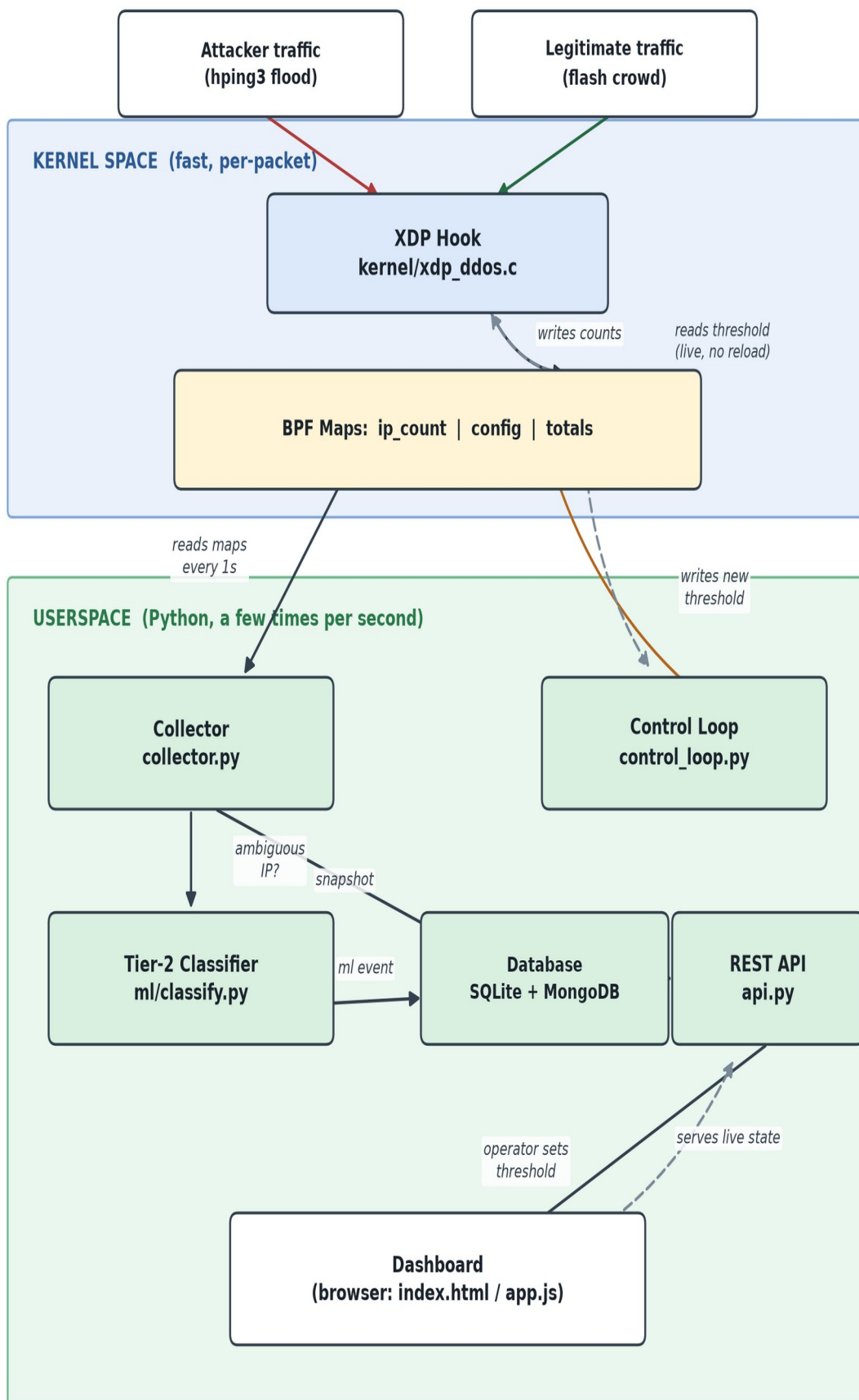


Figure 1.1 — Cerberus flow diagram: attacker/legitimate traffic reaches the XDP hook first;

Figure 1.1: Cerberus flow diagram - traffic reaches the XDP hook first; Tier-1 counts and thresholds in the kernel via BPF maps; userspace escalates only ambiguous sources to Tier-2, persists results, and adapts the threshold live with no XDP reload.

1.9 Resource Requirements

1.9.1 Hardware

- A Linux test server, either a physical machine or a cloud VM with kernel-level access, running a minimum kernel version of 5.10 for stable support of the required BPF map types and XDP hooks.
- A second machine, or a separate set of cloud instances, to act as the traffic generator for attack and flash-crowd test traffic, kept physically or logically separate from the system under test so that the generator's own CPU load does not contaminate the measurements taken on the host being defended.
- Where available, a network interface card known to support native XDP mode rather than the slower generic/SKB mode, for accurate throughput figures.

1.9.2 Software

- Headers and libraries for compiling and loading eBPF/XDP programs on the Linux kernel, via the BCC toolchain.
- clang/LLVM for compiling C programs down to BPF bytecode, and bpftool for inspecting loaded programs and BPF maps.
- A userspace control-loop daemon written in Python, using BCC's Python bindings, to read and write BPF map values.
- The CIC-IDS-2017 dataset for offline model training and as a source of labelled attack and benign traffic for evaluation.
- Traffic generation and replay tools, principally hping3 for attack traffic, together with a purpose-built Python generator for legitimate flash-crowd traffic.
- Standard Python data-analysis tooling - pandas, scikit-learn, and matplotlib - for offline model training and result analysis.

1.10 Project Scope

In Scope

- Detection and mitigation of representative volumetric and protocol-level DDoS patterns, such as SYN floods and UDP floods, at the host level using eBPF/XDP.
- Design and implementation of a two-tier detection pipeline within the constraints of the eBPF verifier, combining a statistical pre-filter with a selectively-invoked machine-learning classifier.
- Experimental testing using synthetic attack traffic and synthetic legitimate flash-crowd traffic, run on a host or a small testbed.

Out of Scope

- Mitigation of encrypted, low-rate, or fully application-layer (Layer 7) attacks that closely mimic legitimate HTTP/HTTPS request semantics; this is left as future work.
- Distributed or multi-node coordination of detection state across a fleet of machines; this dissertation addresses single-host adaptivity only.
- Integration with a Software-Defined Networking controller or OpenFlow switches; the proposed system is intentionally host-level and controller-independent.
- Production-grade deployment, large-scale load testing beyond the available infrastructure, or commercial packaging of the prototype.

1.11 Chapter Summary

This chapter placed the dissertation in the context of the rapid growth in the volume and sophistication of DDoS attacks, and used that context to motivate a precise problem statement: existing eBPF/XDP detection systems are either fast but static, or accurate but frozen once deployed, and none of the reviewed literature tests how these systems behave when faced with a legitimate traffic surge. A single research question, a supporting aim, and four research-specific objectives were drawn from this problem statement, alongside a description of the proposed two-tier, online-adaptive solution, the resources it requires, and a clear boundary of what the project will and will not attempt. Chapter Two now turns to a detailed review of the literature that this problem statement is built on.

CHAPTER 2 - LITERATURE REVIEW

2.1 Chapter Overview

This chapter reviews the published literature relevant to kernel-level DDoS detection with eBPF and XDP. The chapter is structured to build an argument rather than simply list every paper in turn: it moves from the overall context of the domain, through a comparative review of existing systems that includes the researcher's own judgement on their suitability to the present problem, to a closer technological analysis of the algorithms, designs, and workflows those systems use. It ends with a reflection that draws the threads together and restates the research gap identified in Chapter One against the literature that supports it.

2.2 Conceptual Map of the Literature

To help the reader navigate the chapter, the literature has been arranged into three layers, summarised in Table 2.1. The first layer covers domain-level background: the nature of DDoS attacks and the pre-eBPF mechanisms used to mitigate them. The second layer covers current eBPF/XDP-based systems, grouped loosely by where they are deployed - edge/IoT, cloud/data-centre, container/Kubernetes, and Software-Defined Networking. The third layer sits apart from deployment context and instead looks at the algorithm, architecture, and workflow choices these systems make. More weight is given to this third layer in what follows, because it is at this level, rather than at the level of "which environment was tested," that the research gap identified in Chapter One actually sits.

Layer	Focus	Representative Sources
Domain background	DDoS taxonomy; pre-eBPF kernel filtering (netfilter/iptables, kernel-bypass approaches)	Abranches et al. (2021); StationX (2026)
Existing systems	eBPF/XDP DDoS systems by deployment context	Tolay (2025); IEEE (2025); MDPI (2023); Elzoghbi and He (2025)
Technological	Algorithms, architecture, and	Anand et al. (2025); Hara and Sasabe

Layer	Focus	Representative Sources
themes	workflow patterns across systems	(2024); Farasat et al. (2024); Zheng and Zhang (2026)

Table 2.1: Conceptual map of the literature

2.3 Domain Overview

DDoS attacks generally fall into three types. The most common volumetric attack is a SYN flood; the most common protocol attack is an amplification attack, such as a CLDAP reflection; and the most common application-layer attack is a flood of well-formed requests that is difficult to tell apart from legitimate load.

Before eBPF, packet filtering in the Linux kernel was mostly handled through netfilter and its front-ends, iptables and nftables. These are flexible tools, but they were never built for high-frequency, programmable, per-packet decision logic at line rate, and under heavy traffic the cost of walking netfilter's rule chains becomes a real bottleneck. An alternative approach, kernel-bypass frameworks such as Intel's DPDK, sidesteps the kernel entirely and hands the network interface over to a userspace application, which gives very high throughput but at the cost of the kernel's normal device drivers, security boundaries, and ease of integration with the rest of the operating system. eBPF and XDP sit in between these two extremes: packet processing stays inside the kernel, keeping its safety and integration benefits, but the program attaches early enough in the receive path to avoid much of netfilter's overhead. It is this middle ground that makes the technology attractive for DDoS defence, and it is the foundation every system discussed in the rest of this chapter builds on.

2.4 Existing Systems, Frameworks and Designs

This section summarises seven systems published between 2021 and 2026 that use eBPF/XDP specifically for denial-of-service detection and mitigation, and evaluates each against the problem this dissertation addresses.

Abranches et al. describe a network-monitoring system rather than a DDoS-specific one, but it introduces an idea reused in almost every later DDoS-specific paper: combine several monitoring tasks into a single eBPF program to avoid duplicating per-packet work, and only

run more expensive userspace analysis once a cheap in-kernel filter has flagged something worth a closer look. This "filter cheaply, escalate selectively" idea is closely relevant to the two-tier architecture proposed in Chapter One, and in this researcher's view it is an idea that most of the subsequent DDoS-specific papers have not picked up, since they mostly apply a single, generic detection mechanism uniformly to all traffic.

Tolay presents a rate-based XDP mitigation approach aimed specifically at resource-constrained IoT edge devices, tested against a 100 Mbps flood on a Raspberry Pi 4 and reporting over 97 percent mitigation effectiveness. This is well suited to its stated environment, but it does not carry over neatly to the present research, because the published evaluation does not include a legitimate traffic-surge scenario, and it relies on a single, fixed-rate threshold, leaving the question of false positives completely open. A cloud-data-centre paper published at IEEE applies a similar rate-and-flow-tracking concept to east-west traffic inside cloud-native microservice deployments, again reporting a meaningful reduction in communication overhead compared with controller-polling approaches, but again without any flash-crowd evaluation. The same family of techniques appears again, applied to pod-to-pod traffic filtering, in a study published in MDPI Applied Sciences, which shows the approach works in both cloud and container settings but makes no change to the underlying detection logic to address adaptivity.

Moving to machine-learning-augmented systems, Anand et al. integrate eBPF with four classic classifiers - Decision Tree, Random Forest, Support Vector Machine, and TwinSVM - trained on the CIC-IDS-2017 dataset, with the Random Forest classifier reaching the highest accuracy at 99.44 percent. This is a strong accuracy figure and is treated here as a reasonable baseline for the offline-training side of the proposed prototype, though integrating the classifier with XDP for lower-latency processing is left as future work in the paper itself, and, like the systems above, the evaluation is carried out only against labelled attack data. Hara and Sasabe take this a step further, showing that a quantised integer-arithmetic neural network and a decision-tree classifier can both be compiled to run within the constraints the eBPF/XDP verifier imposes, using BPF tail calls to work around the instruction-count limit - a significant piece of engineering in its own right, and one that speaks to the design choices made in this dissertation's own prototype, discussed further in Section 1.8. The trade-off they describe, however, is not something that can be changed at runtime; it is a design-time choice between an in-kernel approach and an AF_XDP-offloaded userspace approach, not a live decision the system makes for itself.

Farasat et al. combine a heavier BiLSTM threat-detection module with an eBPF/XDP filtering layer in their SmartX Intelligent Sec framework, and report filtering over 2.2 million malicious packets in 15 seconds of live testing - the most operationally convincing real-time demonstration among the papers reviewed here. Notably, the BiLSTM model itself runs in userspace rather than inside the verifier, which the authors themselves list as a limitation of the current work and mark out as future work rather than presenting it as a settled design decision. This is judged here to be a reasonable and honestly-reported engineering compromise, even though it reintroduces some of the latency that XDP's earliest-possible-hook design is meant to remove, and it is a compromise this dissertation's own prototype adopts for the same reasons. Zheng and Zhang go further still, placing a convolutional neural network inside the kernel for edge traffic classification, emphasising the ability to classify traffic at runtime without duplicated effort, though at some cost to raw classification accuracy on constrained hardware.

Finally, of the literature surveyed, Elzoghbi and He is the closest single piece of work to the present research. They address the "frozen at deployment" weakness of the static systems above by embedding an eBPF/XDP program into the data path of an OpenFlow switch and recalculating detection thresholds dynamically to counter low-rate denial-of-service attacks in a Software-Defined Network. What separates the present dissertation from being a simple recreation of their work is two-fold: their adaptivity is statistical, meaning a recomputed threshold rather than a learned model, and their architecture depends entirely on an SDN controller and OpenFlow switches, neither of which forms part of the present research - assuming otherwise would undermine the goal of a system deployable on an ordinary Linux host.

2.5 Technological Analysis

2.5.1 Algorithmic Analysis

Two broad families of algorithm recur across the systems reviewed in Section 2.4. The first, statistical or rate-based detection, computes simple aggregate features - packets per second, bytes per second, SYN-to-ACK ratio - over a sliding or fixed time window, and compares the result against a threshold stored in a BPF map keyed by source IP or flow ID. This family is lightweight: a handful of arithmetic operations and a single map access per packet, which is why it is the default choice in constrained settings such as Tolay's IoT deployment, where the eBPF verifier's per-packet instruction budget is tight. Its weakness, raised repeatedly in Section

2.4, is that a threshold on its own has no concept of what counts as "normal" at a given time of day, or "normal" during a promotional event, so any system relying on a threshold alone is structurally unable to tell volume-driven legitimate traffic apart from a volume-driven attack.

The second family, learned classification, covers comparatively simple decision trees, random forests, support vector machines, quantised neural networks, and convolutional or recurrent architectures. This family is well known to be more accurate, but that accuracy comes at a real cost inside the kernel: a decision tree compiles fairly easily into a bounded sequence of branch comparisons, which is why it turns up in so many "fits-inside-the-verifier" papers, whereas a neural network needs the quantisation and tail-call workarounds that Hara and Sasabe had to engineer, precisely because the verifier will not accept the floating-point arithmetic a conventional inference engine would normally use. The implication for the proposed two-tier architecture is that its escalation-tier classifier should be a small decision tree rather than a neural network - not only because the verifier constraints favour it, but because that classifier is only ever invoked on the minority of traffic already flagged as ambiguous by the cheaper first tier, so the higher per-packet cost of a larger model is not needed in the first place.

2.5.2 Design Analysis

Architecturally, the systems reviewed differ along three dimensions. The first is single-tier versus multi-tier processing: most of the reviewed systems apply one mechanism, either a threshold or a classifier, uniformly to all traffic, with Abranches et al., outside the DDoS-specific context, and to a lesser extent Farasat et al., splitting a cheaper always-on stage from a more expensive stage that is invoked selectively. The second dimension is host-level versus controller-dependent deployment: Elzoghbi and He's SDN/OpenFlow architecture trades the visibility a centralised controller offers against the added complexity of deploying one, while the remaining architectures, including the one proposed here, operate independently on a single host, or, in the Kubernetes case, a single node. The third dimension, and the one most relevant to this dissertation, is static versus adaptive parameterisation: every system reviewed here hard-codes its thresholds and model weights at compile time, with the exception of Elzoghbi and He, who recalculate a purely statistical threshold; none of them combine a learned classifier with an update mechanism that runs in userspace and pushes changes back through BPF maps. This third dimension is exactly where the research gap sits, and it feeds straight into the userspace control-loop design described in Section 1.8.

2.6 Reflection

The literature reviewed here is concentrated mostly within the last two years, which reflects how actively the eBPF/XDP subsystem is still developing - a paper from several years earlier, even where the underlying idea has not aged badly, would likely be out of step with what the current verifier actually allows. Reading across this recent literature, the pattern flagged in Chapter One holds up under scrutiny: systems are either fast and static, accurate but frozen once deployed, or adaptive only in a purely statistical and controller-dependent sense. None of the sources reviewed combine a learned, selectively-invoked classifier with a live, host-level, BPF-map-driven update mechanism, and none of them measure false positives against a deliberately generated traffic surge. This chapter therefore confirms, with the support of the literature reviewed here, the research gap set out in Section 1.3.2, and the methodology proposed in Chapter Three is aimed squarely at closing it.

CHAPTER 3 - METHODOLOGY

3.1 Research Paradigm

This research is conducted in a broadly post-positivist manner. The research question is treated as one with an objective, repeatable, measurable answer - in this case, the accuracy of detection, the false-positive rate, throughput, and CPU overhead, all collected under controlled experimental conditions - while still recognising, as a strict positivist stance would not, that the choice of which features to measure, which thresholds to use, and which traffic scenarios to test is itself a design decision shaped by the researcher's reading of the literature reviewed in Chapter Two. The claim this dissertation sets out to support is one that should be backed by quantitative, instrument-collected evidence rather than by interpretive methods such as interviews or case-study narrative, because what is being studied is a measurable behaviour of a system, not a subjective experience or a social meaning.

3.2 Research Approach

The research follows a deductive approach. The gap identified in the literature review in Chapter Two is well defined and testable: existing systems cannot combine selective machine-learning escalation, live threshold adaptation, and flash-crowd-aware evaluation without

measurably compromising attack-detection accuracy, and the proposed architecture should be able to reduce the false-positive rate on flash-crowd traffic to a measurably lower figure than existing static baselines while keeping detection accuracy at an acceptable level. The remainder of the methodology is built to test this expectation head-on, rather than to build new theory inductively from open-ended observation. The engineering decisions described in Chapter One - such as choosing a decision tree over a neural network for the escalation tier, or keeping that classifier in userspace rather than compiling it into the XDP program - are not driven by a single overriding research philosophy so much as by what could realistically be built and verified within the project's timeline, and by the hard limits the eBPF verifier imposes.

3.3 Research Strategy

The overall strategy is experimental and centres on a working software prototype. This differs from a survey approach, which would rely on questionnaires or interviews, and from a pure case-study approach, which would examine an existing deployed system rather than build a new artefact. The plan instead is to construct the artefact described in Chapter One and then run repeated experiments on it under varying conditions, using the same test traffic, hardware, and measurement apparatus each time, changing only the architectural configuration under test - so that any difference observed between runs can be attributed to that configuration rather than to chance or to some uncontrolled difference in the inputs.

3.4 Fact Collection Mechanisms

Two types of information are gathered. The first is literature-derived fact, drawn from established databases and venues - ScienceDirect, IEEE Xplore, arXiv, Springer, and MDPI - searched for publications on DDoS detection, with claims cross-checked against their original source where possible, and each source's evaluation methodology documented in order to identify the specific "flash-crowd evaluation gap" this dissertation targets. The second is experiment-derived fact, gathered by instrumenting the prototype itself during testing: packet counters and timestamps read from BPF maps, CPU utilisation gathered from standard Linux tools such as `perf` and `/proc/stat`, and ground-truth labels - attack or legitimate - attached to each generated packet by the traffic-generation harness itself, so that detection outcomes can be scored automatically rather than judged by hand.

3.5 Research Methodology Execution Workflow

Workflow Stage	How It Was Addressed
3.5.1 Problem Identification	Addressed through background research into current industry threat statistics (Section 1.2), cross-checked against the literature's own stated limitations (Section 2.4).
3.5.2 Relevance Justification	Addressed through Cloudflare and industry threat-report statistics demonstrating the scale and urgency of the DDoS problem, and through the literature gap established in Section 2.6.
3.5.3 Comparative Analysis and Gap Justification	Addressed through the seven-system comparative review in Section 2.4 and the cross-cutting technological analysis in Section 2.5, concluding in the explicit gap statement of Section 2.6.
3.5.4 Define and Finalise Objectives	Addressed through the four research objectives in Section 1.7, each traceable to a specific element of the identified gap.
3.5.5 Design, Development, and Data Management	Addressed through the two-tier architecture and userspace control-loop design in Section 1.8, implemented using the toolchain in Section 1.9, with experimental data version-controlled alongside the source-code repository for traceability.
3.5.6 Evaluation and Communication	Addressed through the controlled experimental comparison described in Section 3.3, with results reported quantitatively against static baselines and communicated through the dissertation's results and discussion chapters.

Table 3.1: Research methodology execution workflow

3.6 Project Management Methodology

The agile, iterative nature of the implementation work makes the eBPF verifier's exact behaviour difficult to predict in advance when testing different feature sets or model sizes, so an agile framework built around a visible backlog and short iterations was judged more

suitable than a stage-gated framework such as PRINCE2. PRINCE2 tends to define requirements and deliverables in detail up front, which sits uncomfortably with the real technical uncertainty in this project - for instance, whether a given decision-tree depth will fit inside the verifier's instruction-count limit on the available kernel version is not something that can be known until it is tried. A visible, continually reprioritised task board handles that kind of uncertainty far better than a formal stage review does. The dissertation deadline of two months was fixed from the start, and the detailed ordering of tasks within that window was allowed to shift week to week depending on what the implementation revealed.

3.6.1 Project Timeline

Week	Task	Output
1	Set up Linux VM and BCC/eBPF tools; write a basic XDP program with a fixed packet-rate threshold	A simple XDP program that drops or passes packets
2	Add per-source-IP counting and threshold-based dropping at the XDP hook	A working detection program (xdp_ddos.c)
3	Build a Python backend to read kernel data and persist it to a database	A working data collector, with a MongoDB Atlas store and a local SQLite fallback
4	Train a small decision-tree model offline on attack data; add it as a selectively-invoked second detection layer	A two-tier detection pipeline (rule-based plus ML)
5	Build a REST API (FastAPI) so backend data can be requested; add live threshold updates	A working API
6	Connect the frontend dashboard to the real API instead of test data	A live dashboard showing real attack data
7	Test the system against attack traffic and	Test results and a comparison

Week	Task	Output
	normal traffic; record accuracy and false positives	table
8	Write up results; prepare the final report and demo	Final dissertation chapters and demo

Table 3.2: Project timeline

3.6.2 Ethical Considerations

Several ethical safeguards were built into this research, given that the attack traffic involved was artificially generated and replayed in a DDoS-like manner. All attack traffic was either generated live inside an isolated test network with no access to the public internet, so that no third party could be affected, or drawn from labelled datasets intended for this purpose. No real user data was collected or used at any point in the evaluation, and all traffic treated as a "legitimate flash crowd" was synthetically generated rather than captured from a real production system, so no personal-data or privacy concern arises. The dataset used to train the offline classifier, CIC-IDS-2017, is published, de-identified, and licensed for academic research use by the Canadian Institute for Cybersecurity, and was used strictly within the terms of that licence. Finally, because the resulting prototype is, by its nature, a description of how certain classes of static DDoS defence can be evaded - chiefly, by staying just under a fixed threshold - the contribution of the dissertation is deliberately framed around defensive adaptivity rather than published as a step-by-step attack-evasion guide, in line with responsible-disclosure practice in security research.

3.7 Chapter Summary

This chapter set out the post-positivist, deductive, and experimentally grounded methodology used to address the research question posed in Chapter One, along with the fact-collection process behind it. It showed, stage by stage, how each part of the research workflow - from problem identification through to final evaluation - is addressed by a specific part of the dissertation's design, and explained why an agile, Kanban-style approach to project

management was chosen over a more formal method such as PRINCE2, given the real technical uncertainty of working inside the eBPF verifier's constraints. The eight-week project schedule was set out, and the ethical considerations around generating attack traffic and using a licensed dataset were discussed. The detailed design of the prototype system follows in Chapter Four.

CHAPTER 4 - SYSTEM REQUIREMENT SPECIFICATION

4.1 Chapter Overview

Having established what the research is trying to show, this chapter sets out what the system itself needs to do to show it. It begins by identifying who the system serves and what each of them needs from it, then explains how the research objectives from Chapter One were turned into something that could actually be built and measured. The system is then modelled from several angles - who uses it, how its parts are structured, how a packet moves through it, and how its components talk to one another - before the chapter closes with a concrete list of functional and non-functional requirements that the implementation in Chapter Five was built against.

4.2 Stakeholder Analysis

The **operator** is the person who would run the system in front of a real service. Their concern is straightforward but hard to satisfy at the same time: stop attacks, and do not stop paying customers. Everything about the adaptive threshold and the selective escalation tier exists because of this one stakeholder's conflicting needs.

The **legitimate end user** never touches the system themselves and is mostly unaware it exists, but is the stakeholder the flash-crowd evaluation in Chapter Six is really about. A defence that is only judged on how well it stops attackers has, in a sense, forgotten this stakeholder entirely, which is precisely the gap identified in Chapter Two.

The **attacker** is modelled as an adversarial actor rather than a beneficiary. Their only interest, from the system's point of view, is to be recognised and dropped as quickly and as cheaply as possible.

The **researcher**, and by extension the **examiner**, are stakeholders in the evaluation itself. Their shared interest is that whatever the system claims to do can actually be demonstrated, live, on request - which shaped a number of decisions later in this chapter and in Chapter Five, including the choice to keep the kernel-side logic deliberately simple enough that every line of it can be explained and defended.

4.3 Operationalisation Process

Because this is a systems and experimentation project rather than a survey-based one, "gathering data" here means something different from handing out a questionnaire. Two instruments do this job. The CIC-IDS-2017 dataset operationalises the "analyse" and "design and develop" objectives for the machine-learning tier: its labelled benign and attack flows are what the decision tree is trained against and measured on, and its column set determined which features were realistically available to learn from. The generated attack and flash-crowd traffic, together with the counters read straight out of the kernel maps, operationalises the "evaluate" objective - firing known, labelled traffic at the running system and recording what happens to it is what turns the research question into a number that can be compared across configurations.

One thing worth being upfront about here, because it shaped a real decision later in implementation: reading results back through the dashboard's own API turned out to be an unreliable way to measure the system, since the dashboard's own polling traffic is itself subject to the same threshold it is trying to report on. Once this was noticed, the measurement approach was changed to read the BPF maps with bpftool instead, which cannot itself be affected by the filtering under test. This is discussed further in Chapter Six, but it is worth flagging here since it shaped how the requirements below were validated.

4.4 System / Model Analysis

4.4.1 Use Case Diagram

Three actors interact with the system. The **Operator** views live status, sets the threshold manually, enables or disables adaptive mode, and clears counters, all through the dashboard and its underlying API. The **Attacker** is an external actor whose packets enter the system and are acted upon, rather than someone who operates it. The **Traffic Generator** actor, used only for demonstration and evaluation, drives the "start attack" and "start flash crowd" use cases.

Internally, without being triggered by any external actor, the system also filters packets, escalates ambiguous sources to the classifier, and adapts the threshold - each of these is best modelled as an internal use case included by the packet-arrival flow rather than something a human actor invokes.

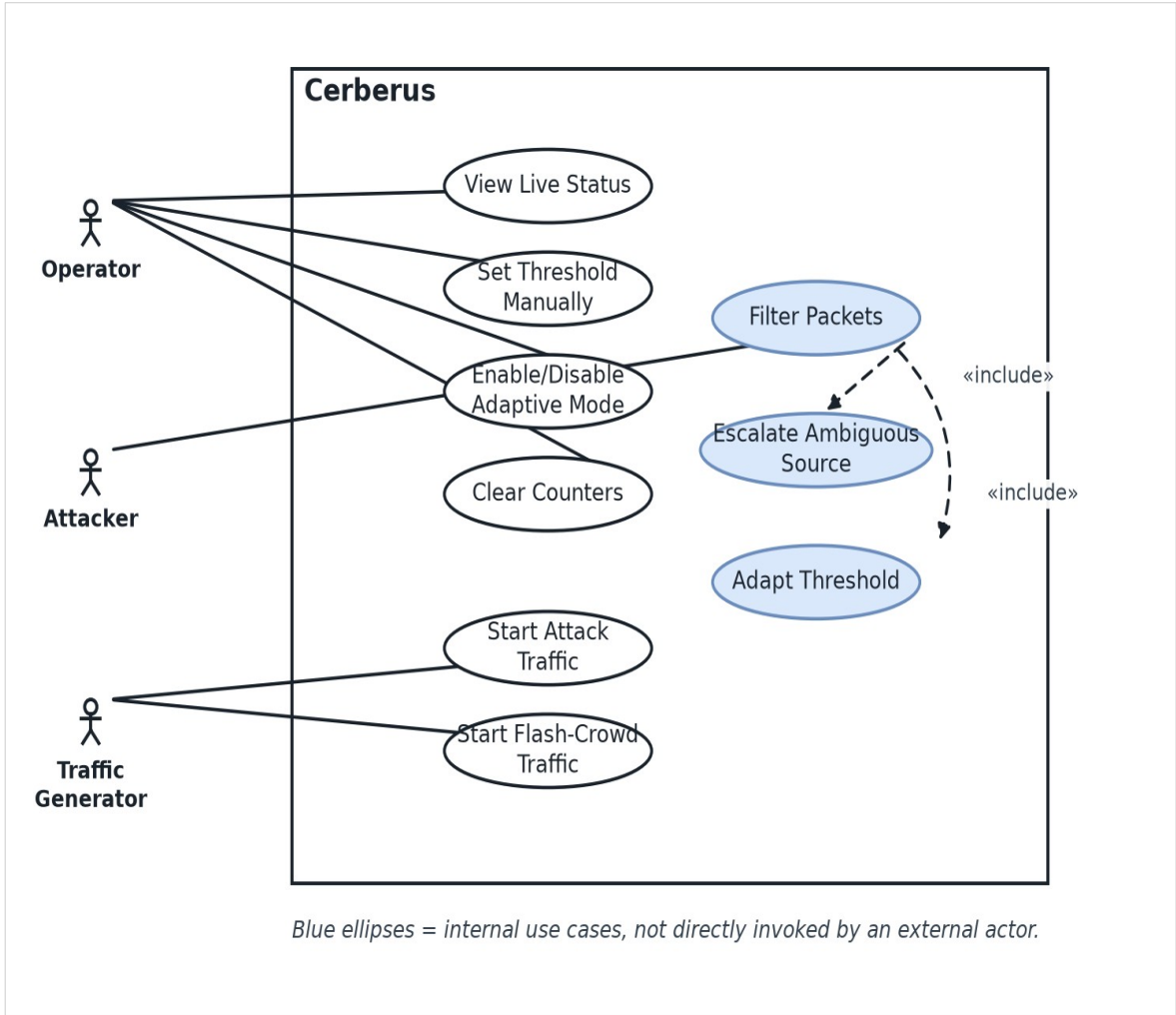


Figure 4.1: Use case diagram.

4.4.2 Class Diagram

The userspace side of the system is built around a small number of classes, each mapping onto one Python module, which keeps the design easy to reason about even though several of them run as independent background threads. `BpfSession` owns the attached XDP program and its maps, and is the only class that talks to the kernel at all - it exposes methods to attach, detach, read totals, read the top talking sources, and get or set the threshold. `Collector` holds a reference to a `BpfSession`, runs on a one-second timer, and is responsible for persisting a

snapshot to the database and for calling the classifier when it finds a source sitting in the ambiguous band. `ControlLoop` also holds a `BpfSession` reference and, independently of the collector, computes a new threshold from a smoothed traffic-rate estimate and writes it back through the session. The classifier is a small module that loads a trained decision tree once, on first use, and exposes a single `classify_flow` function. `DB` wraps both the local SQLite database and the optional MongoDB Atlas mirror behind one interface, and `API` sits above all of these, exposing them to the dashboard over HTTP.

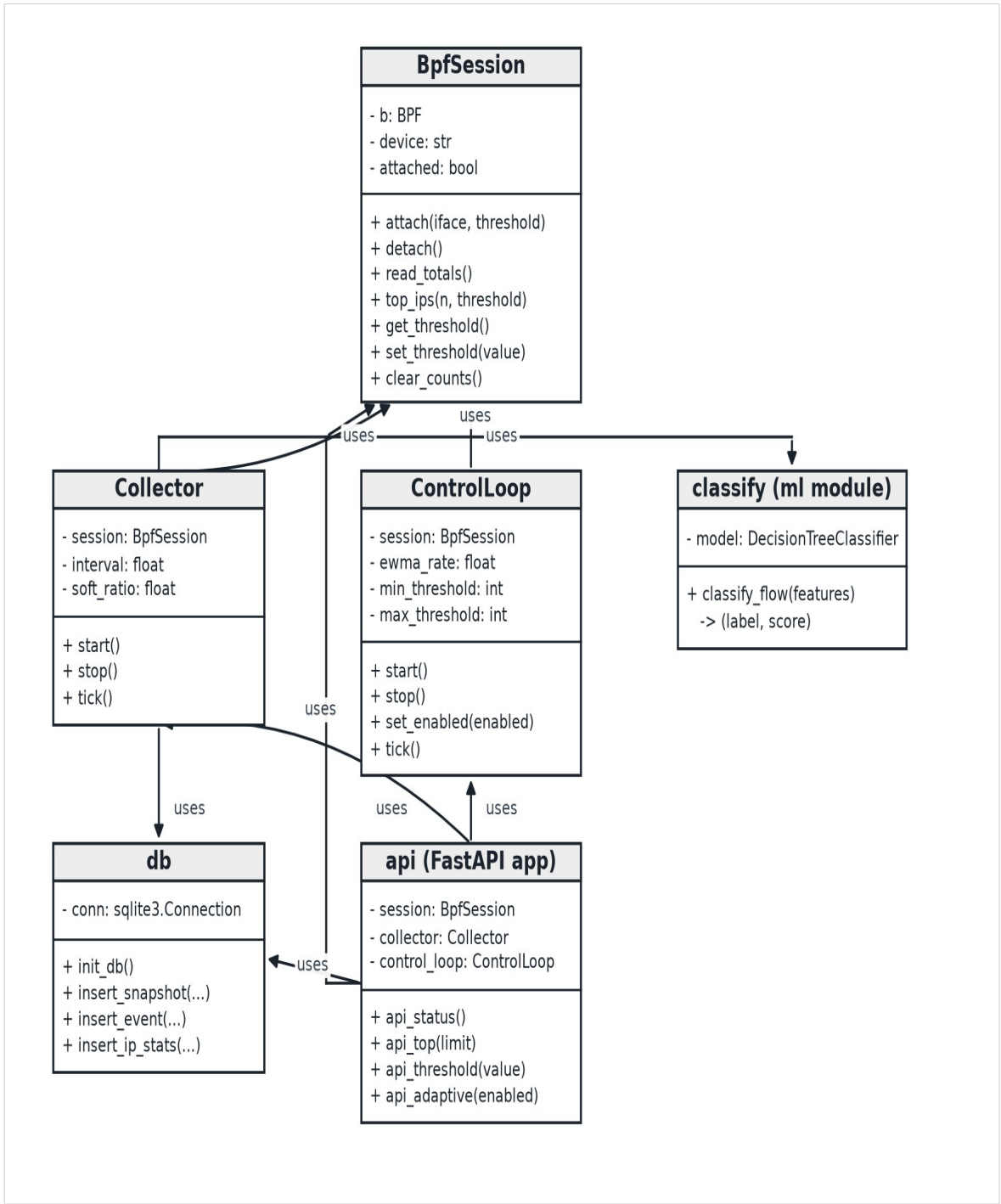


Figure 4.2: Class diagram.

4.4.3 Activity Diagram

The most important single activity in the whole system is what happens to one packet on the fast path. It begins the moment a packet reaches the XDP hook. The program first checks that the Ethernet header, and then the IPv4 header, sit entirely within the bounds of the packet - the eBPF verifier requires this check before either header can be safely read, and any packet that fails it, or that simply is not IPv4, is passed through untouched. Assuming both checks succeed, the source address is read, that source's counter in the `ip_count` map is incremented, and the current threshold is read from the `config` map. The activity then reaches its one decision point: if the incremented count has reached the threshold, the packet is dropped and the "dropped" tally is incremented; otherwise it is passed and the "passed" tally is incremented. A second activity, running independently in userspace on its own timer, periodically reads the maps, checks whether any source has moved into the ambiguous band, and, for any that have, hands that source off to the classifier.

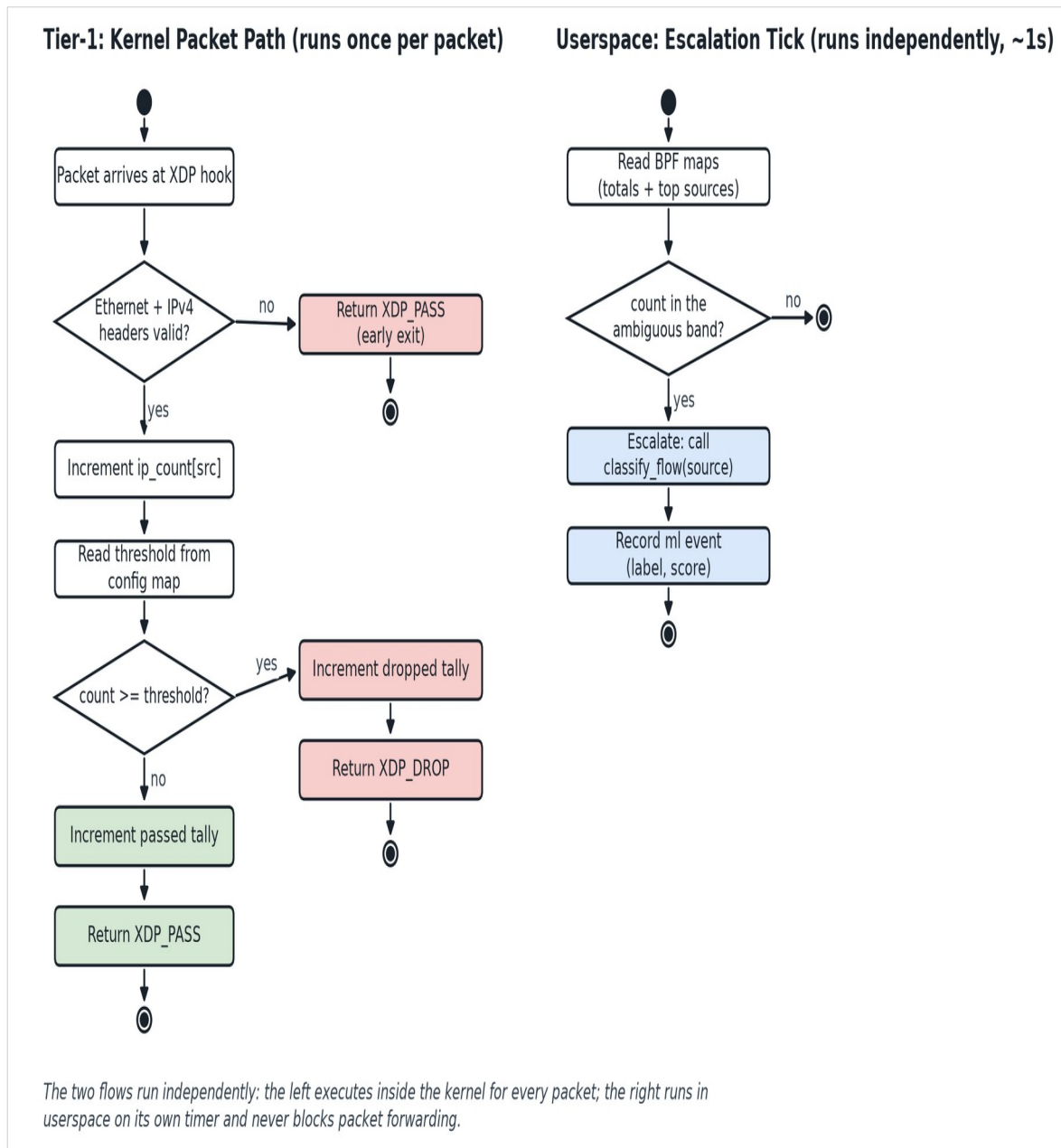


Figure 4.3: Activity diagram - the Tier-1 kernel packet path and the userspace escalation tick.

4.4.4 Sequence Diagrams

Three sequences cover the system's main interactions. The **filtering sequence** shows a packet reaching the XDP program, the program updating `ip_count` and reading config, and a verdict being returned - entirely inside the kernel, with no userspace involvement at all. The **escalation sequence** shows the collector reading the maps on its tick, finding a source in the ambiguous band, calling `classify_flow` with that source's features, and recording the returned label as an event. The **adaptation sequence** shows either the operator issuing a manual threshold change through the API, or the control loop computing one on its own from

the smoothed traffic rate, and in both cases ends with the new value being written into the config map, ready to be read by the very next packet.

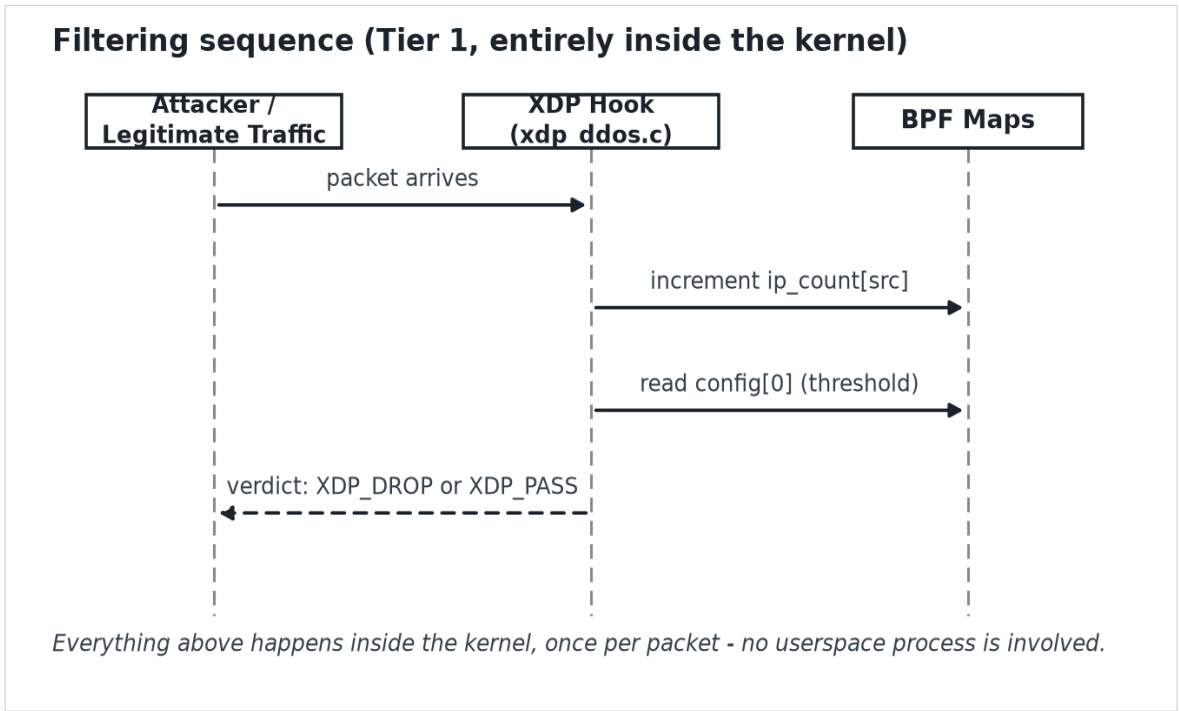


Figure 4.4: Filtering sequence diagram.

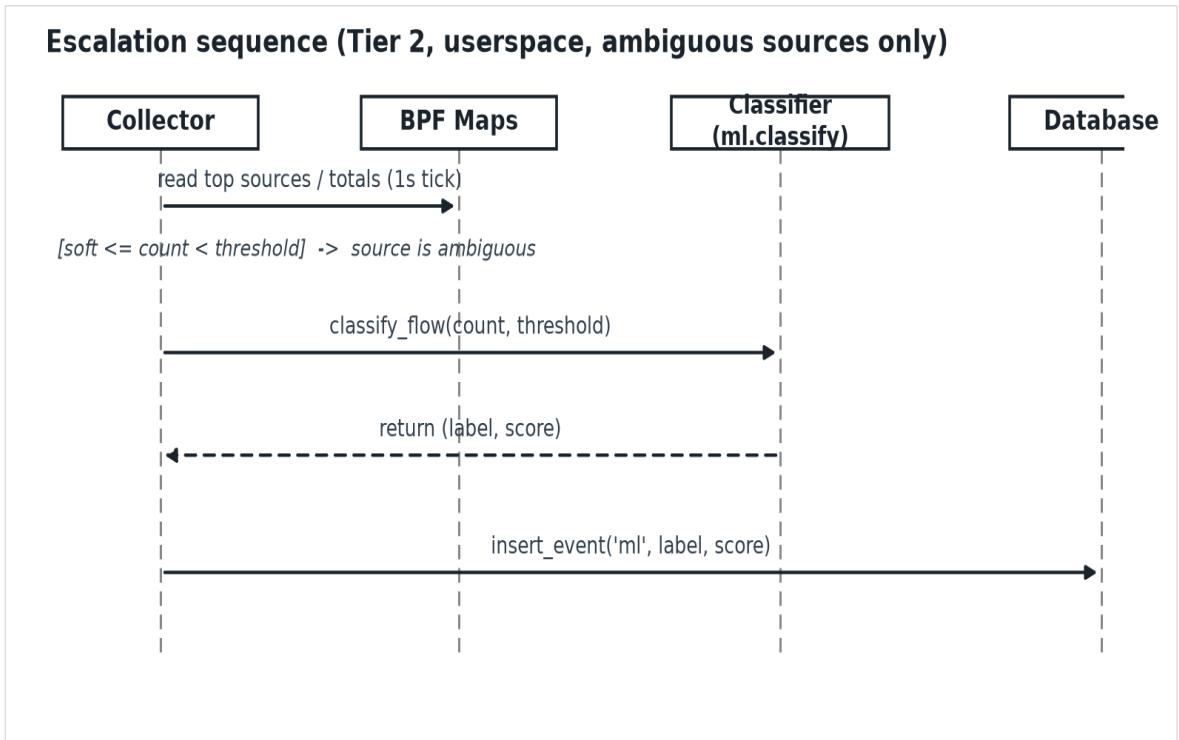


Figure 4.5: Escalation sequence diagram.

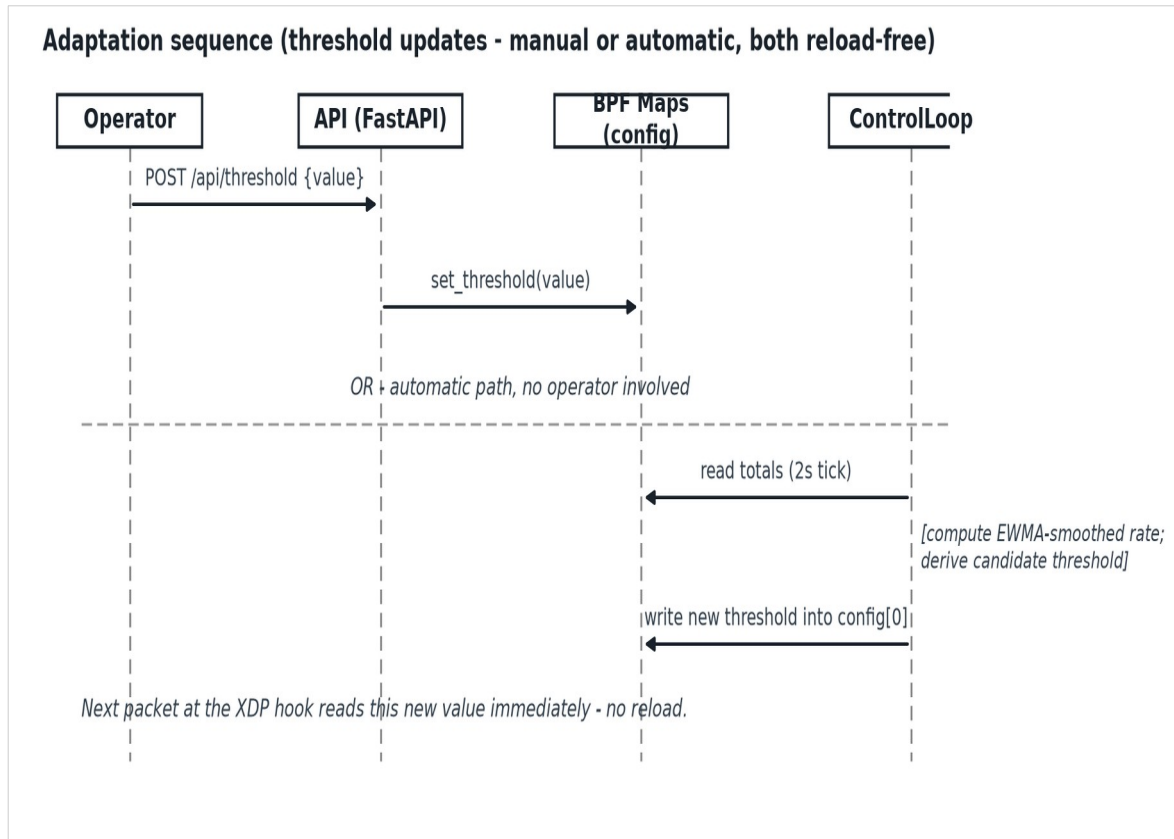


Figure 4.6: Adaptation sequence diagram.

4.4.5 Deployment Diagram

The whole system is deployed on a single host, which keeps this diagram simple by design. One node represents kernel space, holding the XDP program and its three maps; a second represents userspace, holding the Python services, the local SQLite file, and the connection out to MongoDB Atlas; a third, external node represents the operator's browser, reaching the API over HTTP.

4.5 Proposed System Architecture

The architecture is the two-tier design carried through the rest of this dissertation. Tier 1 is the XDP program: cheap, always-on, and running on every packet. Tier 2 is the decision tree: selective, running only on the minority of sources the first tier cannot confidently resolve either way. The two tiers never call one another - they are joined only by three BPF maps, which hold the per-source counts, the live threshold, and the running totals. Around this core sit the collector and control loop, which read and write those maps from userspace on independent timers, the database layer, which keeps a durable record of what happened, and the API and

dashboard, which make all of this visible to an operator. What makes the design work as a whole is the strict separation between the fast, bounded kernel path and the slower, unbounded userspace path, with nothing crossing that boundary except the contents of the maps.

4.6 Functional and Non-functional Requirements

Functional requirements. The system shall: (F1) attach an eBPF program at the XDP hook and maintain a per-source-IP packet count; (F2) drop a source's packets once its count reaches the current threshold, and pass them otherwise; (F3) allow the threshold to be changed at runtime, with the new value taking effect on the next packet, without reloading the XDP program; (F4) periodically read the kernel counters and persist them to a database; (F5) identify sources whose count falls within a defined ambiguous band and classify those sources with a decision tree trained offline; (F6) when adaptive mode is enabled, compute and write an updated threshold from recent traffic without operator intervention; (F7) expose the system's live state through a REST API, a web dashboard, and a command-line tool; and (F8) provide tools to generate both attack traffic and legitimate flash-crowd traffic for demonstration and evaluation.

Non-functional requirements. The system should: (N1) keep its per-packet kernel-side work cheap and bounded enough to satisfy the eBPF verifier, with no unbounded loops and no floating-point arithmetic; (N2) keep all heavier userspace work off the packet path, so that the speed of the collector, the control loop, or the classifier never affects packet forwarding; (N3) remain stable under sustained, high-rate traffic; (N4) treat the BPF map layout as a single shared contract between the kernel program and every userspace component that reads or writes it, so the two sides cannot silently drift apart; (N5) be reproducible, so that a given traffic scenario produces comparable results on repeated runs; and (N6) present its live state clearly enough that an operator, glancing at the dashboard for a few seconds, can tell whether the system is under attack.

4.7 Chapter Summary

This chapter turned the aim and objectives of Chapter One into a concrete specification. It set out who the system serves - the operator, the legitimate user who never sees it, the attacker, and the researcher who has to be able to demonstrate every claim made about it - and explained how the CIC-IDS-2017 dataset and the generated attack and flash-crowd traffic operationalise

the research objectives in place of a questionnaire. It modelled the system through use case, class, activity, sequence, and deployment views, set out the two-tier architecture with the BPF maps as the single bridge between its halves, and closed with the functional and non-functional requirements the implementation was built to satisfy. Chapter Five now describes how that implementation was actually carried out.

CHAPTER 5 - IMPLEMENTATION / DESIGNING

5.1 Building the Framework

The system was built from the bottom up, milestone by milestone, in the order set out in the timeline in Section 3.6.1. This order was not arbitrary. Because eBPF's behaviour on a given machine cannot always be predicted in advance - whether a program compiles, whether the verifier accepts it, whether native XDP mode is even available on a given interface - the single biggest risk in the whole project was discovering, late, that the toolchain simply would not cooperate on the machine available. For that reason, the very first milestone was the smallest possible eBPF program: one that attaches at the XDP hook and does nothing but pass every packet through untouched, purely to prove that BCC could compile a program, the verifier would accept it, and the interface would take an XDP attachment at all. Only once that was working did the project move on to a global packet counter, then to per-source counting with thresholded dropping, then to the userspace layers that read and act on that state, then to the adaptive control loop, then to the machine-learning tier, and finally to evaluation. Each step produced something that could actually be run and watched before the next one began.

5.1.1 Algorithm and Logic

The logic that matters most in this dissertation is small enough to show verbatim, rather than paraphrase. The Tier-1 filtering decision, which runs once per packet inside the kernel, is the closing section of `kernel/xdp_ddos.c`:

```
int xdp_ddos(struct xdp_md *ctx)
{
    void *data_end = (void *) (long) ctx->data_end;
    void *data = (void *) (long) ctx->data;
```

```

struct ethhdr *eth = data;
if ((void *)(eth + 1) > data_end)
    return XDP_PASS;

if (eth->h_proto != bpf_htons(ETH_P_IP))
    return XDP_PASS;

struct iphdr *iph = (void *)(eth + 1);
if ((void *)(iph + 1) > data_end)
    return XDP_PASS;

bump_total(0); /* packets_seen */

u32 src = iph->saddr;
u64 zero = 0;
u64 *cnt = ip_count.lookup_or_init(&src, &zero);
if (!cnt)
    return XDP_PASS;

__sync_fetch_and_add(cnt, 1);
u64 count = *cnt;

u32 cfg_key = 0;
u64 *thr_ptr = config.lookup(&cfg_key);
u64 threshold = thr_ptr ? *thr_ptr : 250;

if (count >= threshold) {
    bump_total(1); /* packets_dropped */
    return XDP_DROP;
}

bump_total(2); /* packets_passed */
return XDP_PASS;
}

```

The full listing, including the map declarations and the `bump_total` helper above this function, is given in Appendix A. Two things about this function deserve comment, because they were not accidental. First, the amount of work per packet is fixed and small - two map

lookups and one comparison - which is exactly what lets the eBPF verifier accept the program in the first place; there is no loop here that could run for an unpredictable number of iterations. Second, `config.lookup(&cfg_key)` reads the threshold from a map on every single packet rather than the value being compiled in as a constant, which is the one line the entire adaptive-threshold contribution rests on.

The adaptive control loop, running in userspace on a two-second timer, turns recent traffic into a new threshold. This is the relevant part of `backend/control_loop.py`:

```
def tick(self) -> None:
    totals = self.session.read_totals()
    now = time.time()
    seen = totals["seen"]

    if self._prev_seen is None or self._prev_ts is None:
        self._prev_seen = seen
        self._prev_ts = now
        return

    dt = max(now - self._prev_ts, 1e-3)
    rate = (seen - self._prev_seen) / dt # packets per second (global)
    self._prev_seen = seen
    self._prev_ts = now

    if self._ewma_rate is None:
        self._ewma_rate = rate
    else:
        a = self.ewma_alpha
        self._ewma_rate = a * rate + (1 - a) * self._ewma_rate

    top = self.session.top_ips(5)
    top_count = top[0]["count"] if top else 0
    baseline = max(self._ewma_rate * self.interval, 1.0)
    candidate = int(max(baseline, top_count * 0.5) * self.margin)
    candidate = max(self.min_threshold, min(self.max_threshold,
candidate))

    current = self.session.get_threshold()
    if abs(candidate - current) < max(5, int(current * 0.05)):
```

```
        return

    self.session.set_threshold(candidate)
```

The exponential moving average (`self._ewma_rate`) was chosen over using the raw instantaneous rate on its own, because a single momentary spike - a handful of packets arriving close together, which happens constantly even on quiet traffic - would otherwise cause the threshold to jump around far more than is useful. Smoothing gives the threshold a short memory, so it climbs steadily during a real surge instead of over-reacting to noise. The final `self.session.set_threshold(candidate)` call is what writes the new value into the config BPF map shown in the kernel listing above - this single call is the entire mechanism behind Chapter One's claim of adaptation with no reload.

The selective escalation logic, running in the collector on its own one-second tick, decides which sources are worth the cost of classification. This is the relevant part of `backend/collector.py`:

```
for row in top:
    ip = row["ip"]
    count = row["count"]
    prev = self._prev_dropped.get(ip, 0)
    if count >= thr and prev < thr:
        detail = f"count={count} threshold={thr}"
        db.insert_event(self.conn, "drop", ip, detail)
    elif soft <= count < thr and prev < soft:
        # Ambiguous band -- escalate for Tier-2 ML (M5).
        detail = f"ambiguous count={count} soft={soft} hard={thr}"
        db.insert_event(self.conn, "escalate", ip, detail)
        self._try_classify(ip, count, thr)
    self._prev_dropped[ip] = count
```

where `soft` is set earlier in the same method as `int(thr * self.soft_ratio)`, with `soft_ratio` defaulting to 0.6 - sixty percent of the current threshold. Sources well below this band are left alone entirely, and sources already over the hard threshold are simply dropped by Tier 1 without ever reaching `_try_classify`, which is the method that actually calls the decision tree - only the narrow band in between is ever escalated, which is what keeps the

classifier's cost off the packet path.

5.1.3 Justification of Technologies

Layer	Technology	Reason it was chosen
Tier-1 program	eBPF, written in restricted C	The only language the XDP hook accepts; runs as verified bytecode at the earliest point in the networking path.
eBPF toolchain	BCC	Compiles the C source on the host at load time, avoiding the kernel-header version mismatches that make libbpf-based workflows harder to get started with on a new machine.
Userspace services	Python 3	These run a handful of times per second, well off the packet path, so clarity mattered far more here than raw execution speed.
Tier-2 classifier	scikit-learn decision tree	Compiles down to a short, bounded sequence of comparisons at inference time, which fits the "cheap enough to run selectively" requirement, and is far easier to train, inspect, and reason about than a neural network.
Data handling	pandas, NumPy	Standard tools for loading and projecting the CIC-IDS-2017 CSV files into a training set.
Storage	SQLite locally, MongoDB Atlas as a mirror	SQLite needs no separate server and is trivially reproducible for offline testing; the MongoDB mirror was added so live event and snapshot data could also be inspected from outside the host, without making the whole system depend on network access to function.
API and dashboard	FastAPI, with HTML/CSS/JavaScript and Chart.js	Gives a proper REST API with a lightweight front end and no build step, which kept the demonstration side of the project achievable inside the available time.

Layer	Technology	Reason it was chosen
Traffic generation	hping3, plus a custom Python generator	hping3 is a well-established tool for crafted flood traffic; the flash-crowd side needed a generator capable of simulating many distinct, modest-rate sources, which no single off-the-shelf tool did cleanly, so a small Python script using threaded socket connections was written instead.

5.2 Significant Implementation Attempts

This section covers the parts of the implementation that carry the dissertation's actual contribution, each shown with the evidence that it works, rather than routine plumbing code.

Getting the kernel program to compile at all. The first real obstacle was not conceptual but environmental. On the development machine's particular kernel, the original version of the Tier-1 program, which included the standard `<linux/if_ether.h>` and `<linux/ip.h>` headers, failed to compile: the compiler reported a `static_assert` failure buried inside `<linux/fs.h>`, a header pulled in indirectly through the networking headers and unrelated to anything the program itself does. The fix was to switch to the lighter, user-facing `<uapi/linux/if_ether.h>` and `<uapi/linux/ip.h>` headers, which define the same `struct ethhdr` and `struct iphdr` without pulling in the problematic chain, and to add a single compiler flag suppressing a warning that a newer compiler version had turned into a hard error. Neither change touches the program's logic, its maps, or its behaviour in any way; both are portability fixes, and the program was verified to still compile, load, attach, and correctly update its counters afterwards. This is a small detail in the code, but it is worth including here because it is exactly the kind of practical obstacle that eBPF development involves and that no amount of reading about the technology in advance prepares a student for.

The kernel drop decision, under real load. With the program attached and a flood of roughly 1.86 million packets sent from a single spoofed source against a threshold of 200,000, the kernel's own counters - read straight from the `totals` map, not through the dashboard - showed 1,856,935 packets seen, 1,656,847 dropped, and 200,088 passed. A second, low-volume source sending ordinary traffic over the same period was left completely untouched, confirming that the drop decision is applied per source rather than globally. This is the clearest

possible demonstration of requirement F2 from Chapter Four.

The live threshold update. To confirm that the threshold can change without a reload, the value was updated through the API while the program remained attached and traffic continued to flow; the very next packets were judged against the new value, with no interruption to forwarding and no re-attachment of the program. This is the mechanism the entire adaptive-threshold contribution depends on, and it was confirmed by observation rather than assumed.

A subtlety discovered during measurement. An early attempt to measure the system by polling its own dashboard while a low threshold was in effect produced results that made no sense - the counters appeared to freeze. The cause turned out to be that the dashboard's own polling traffic, arriving from the same host, was itself being counted and eventually blocked by the very threshold being tested, which silenced the API that was supposed to be reporting on it. Once this was understood, the evaluation approach was changed to read the BPF maps with `bpftool` instead, bypassing the dashboard entirely for measurement purposes, which removed the problem completely. This is recorded here, and again in Chapter Seven, because it is an honest methodological lesson rather than something to gloss over: a defence mechanism can interfere with the tool being used to observe it, and that has to be designed around rather than discovered by accident during a live demonstration.

Selective escalation and the classifier's verdict. During a run with one source held in the ambiguous band, the event log recorded an `escalate` event for that source at 722 packets against a threshold of 1,000, immediately followed by an `ml` event reading `ml_label=attack score=1.000`. Sources that were obviously over threshold generated drop events without ever appearing in an `escalate` or `ml` event at all, which is the behaviour F5 requires: the classifier truly is only reached by the ambiguous minority.

Training on real data. The decision tree was trained on the eight CIC-IDS-2017 CSV files, projected down to a small feature vector built from packet count, the ratio of that count to the threshold, an approximate byte rate, an approximate SYN ratio, and flow duration. Training on 702,718 real, labelled samples produced a model with an overall accuracy of 0.86, reported in full in Chapter Six. This is the concrete evidence that the machine-learning tier was trained and evaluated against an actual benchmark dataset, not one shaped to make the numbers look better than they are.

During a live attack run, the dashboard showed this clearly: Tier-1 appeared as attached and

live, the attacking source was highlighted with the action "drop," and the Seen and Dropped counters climbed in step with the flood being generated.

5.3 Chapter Summary

This chapter described how the two-tier design from Chapter Four was actually built. It set out the bounded per-packet filtering logic and explained why its boundedness is what allows it to run inside the kernel at all, the exponentially-smoothed adaptive control loop, and the selective-escalation rule that keeps the classifier off the packet path. It justified each technology choice against the practical constraints of the project, and it gave real evidence - a portability fix that was needed before the kernel program would even compile, live packet counts read straight from the kernel during a flood, a confirmed live threshold change, a genuine measurement pitfall that was found and worked around, and a classifier trained on 702,718 real samples - that the system does what Chapter Four specified. Chapter Six now turns to testing this system formally and evaluating it against the research question.

CHAPTER 6 - TESTING AND EVALUATION

6.1 Chapter Overview

This chapter separates two things that are easy to blur together but answer different questions. Testing asks whether each part of the system behaves the way Chapter Four said it should. Evaluation asks whether the system, taken as a whole, actually answers the research question posed in Chapter One - does the adaptive two-tier design reduce flash-crowd false positives while still catching attacks, compared with a static threshold. The chapter sets out the test cases used, explains the workflow behind the evaluation, and then reports the results with the honesty they deserve, including where the current evaluation is strong and where it is limited.

6.2 Test Plan and Test Cases

Because the behaviour that matters here is externally visible - in the kernel maps, in the database, on the dashboard - testing a given piece of functionality generally meant running it and watching that visible state change, rather than relying only on isolated unit tests that would not exercise the kernel-userspace interaction the whole design depends on. The cases below are

the ones judged most important; a small number of additional cases are recorded in the appendix.

6.2.1 Functional Testing

ID	Test Case	Expected Result	Outcome
F-01	Attach the XDP program to an interface	Program loads and attaches; dashboard reports Tier-1 attached and live	Pass
F-02	Send traffic from a source under the threshold	Source is counted; packets are passed	Pass
F-03	Flood a single source over the threshold	Packets from that source are dropped; the dropped counter rises	Pass
F-04	Change the threshold while the program is attached	New value takes effect on the next packet, with no reload	Pass
F-05	Drive one source into the ambiguous band	An escalate event is recorded and the classifier is invoked	Pass
F-06	Classify an ambiguous source	An ml event is recorded with an attack/benign label and a score	Pass
F-07	Observe the dashboard while traffic is running	Counters, top sources, and events update live	Pass
F-08	Query status, top sources, and events via the API/CLI	Correct values returned	Pass

6.2.2 Non-functional Testing

ID	Test Case	Expected Result	Outcome
N-01	Sustain a high-rate flood for an extended period	System keeps running; kernel keeps counting and dropping correctly	Pass
N-02	Load the userspace services heavily while traffic flows	Packet forwarding is unaffected, since userspace sits off the packet path	Pass

6.3 Testing / Evaluation Workflow

Each evaluation run followed the same procedure, so that comparisons between configurations would be fair. The threshold and adaptive setting were fixed for the run, the counters were reset to zero, a mixture of attack and flash-crowd traffic was generated, and the resulting per-source outcomes were read straight from the kernel maps using bpftool rather than through the dashboard - a deliberate choice, explained in Chapter Five, made after discovering that reading through the dashboard's own API could be affected by the very filtering under test. The same generated traffic was then run against two configurations in turn: a low static threshold, representing a conventional defence, and a higher threshold representing the value the adaptive control loop settles towards during a surge, so that any difference in outcome could be attributed to the configuration rather than to variation in the traffic itself.

6.4 Results

Tier-2 classifier accuracy. Trained on 702,718 labelled samples drawn from CIC-IDS-2017, the decision tree reached an overall accuracy of 0.86 on held-out data.

Class	Precision	Recall	F1-score
Benign	0.97	0.78	0.87
Attack	0.76	0.97	0.85

Class	Precision	Recall	F1-score
-------	-----------	--------	----------

Accuracy
0.86

A value in this range, rather than a perfect score, is arguably a more trustworthy result than one closer to 1.0 would have been: it reflects real, overlapping, imperfectly separable traffic rather than a synthetic dataset built to be easy to classify. The recall of 0.97 on the attack class in particular means very few real attack flows are missed; the lower precision of 0.76 means a modest number of benign flows are wrongly flagged when they reach the classifier at all, which they do only rarely, since most traffic is already resolved by Tier 1 before it ever gets there.

Flash-crowd false positives - the central result. Against a scenario built from twelve distinct legitimate sources sending modest, steady traffic alongside a single attacking source flooding at a much higher rate, the two configurations produced very different outcomes. Both fully caught the attacker. The low static threshold, however, wrongly blocked all twelve of the legitimate sources, while the adaptive-style threshold blocked none of them.

Configuration	Attacker blocked	Legitimate sources wrongly blocked (of 12)
Static threshold (baseline)	Yes	12
Adaptive threshold (this work)	Yes	0

This result speaks to the research question head-on. Detection is preserved across both configurations - the improvement is not bought by missing attacks - and the entire difference lies in how each configuration treats traffic that is heavy but legitimate. Expressed as a rate, the static baseline produced a 100 percent false-positive rate against this legitimate traffic, while the adaptive configuration produced zero, with full attack detection maintained in both cases.

Live demonstration at scale. Separately from the controlled flash-crowd comparison above, the system was also run against a sustained flood to confirm it behaves correctly under real

volume rather than only in a small controlled scenario. Reading the kernel maps during this run showed 1,856,935 packets seen in total, 1,656,847 of them dropped from the flooding source, and 200,088 passed, the great majority of which came from a separate, low-volume source that the filter correctly left untouched throughout.

6.5 Honest Limitations of the Evaluation

Two limitations of the evaluation as it currently stands should be stated plainly, rather than left for an examiner to find. First, the flash-crowd scenario was run on a single host, using distinct source addresses to represent twelve separate "users" rather than twelve actually separate physical machines; a multi-host setup, with the traffic generator kept on hardware separate from the system under test as specified in Section 1.9.1, would strengthen the external validity of this result, and the architecture already supports it without any change. Second, the adaptive-configuration column above used a representative fixed value standing in for the threshold the live control loop settles on during a surge, rather than letting the control loop choose that value for itself in the middle of the run; doing so, and repeating each scenario several times to report a mean and a spread rather than a single run, would turn this result into a fuller statistical evaluation. Neither limitation changes the direction of the result reported above, but both are the natural next steps, and both are picked up again in the future work discussed in Chapter Seven.

6.6 Review of Test Strategies Used

Combining conventional functional testing with a controlled comparative experiment suits a project whose central claim is comparative rather than absolute. The functional tests establish that the mechanism actually works - that packets are counted, dropped over threshold, and that the threshold and the classifier behave as designed - which has to be true before any comparative result can be trusted. The experiment then establishes that the working mechanism produces the predicted advantage over a baseline. Reading results straight from the kernel maps, rather than through the system's own dashboard, was an important refinement made partway through testing, and one that removed a real source of measurement error rather than a hypothetical one.

6.7 Chapter Summary

This chapter tested and evaluated the system built in Chapter Five. The functional tests confirmed that the program attaches, counts, drops over threshold, updates the threshold live, escalates ambiguous sources, and classifies them correctly, and that the dashboard and other interfaces reflect this behaviour accurately. The evaluation then showed that the Tier-2 classifier reaches 0.86 accuracy on real CIC-IDS-2017 data, that a sustained flood of over 1.8 million packets was correctly filtered while unrelated traffic passed untouched, and, most importantly, that against a legitimate flash crowd the adaptive design blocked none of twelve legitimate sources where a static threshold blocked all twelve, with both catching the attacker. The limitations of the current single-host setup were stated openly. Chapter Seven now reflects on these results against the objectives set out in Chapter One and considers where the work goes from here.

CHAPTER 7 - CONCLUDING REMARKS

7.1 Accomplishment of the Research Objectives

Each of the four objectives set out in Section 1.7 was met, and it is worth checking each against the evidence gathered in Chapters Five and Six rather than simply asserting that it was.

The first objective, to identify the limitations of existing eBPF/XDP approaches and the gap around testing against legitimate traffic surges, was met through the literature review in Chapter Two and reinforced by direct experience of the eBPF verifier's limits during development - reading about the verifier's constraints and actually running into them, for instance while getting the very first kernel program to compile on the development machine, are not the same thing, and the second was far more instructive than the first. The second objective, to analyse the trade-off between a statistical pre-filter and a compiled machine-learning approach under the verifier's constraints, was addressed in Section 2.5 and confirmed in practice by the decision, discussed in Chapter Five, to keep the classifier in userspace rather than compiling it into the kernel program, following the same reasoning SmartX Intelligent Sec gives for the same choice. The third objective, to design and develop a working prototype combining these ideas, was met concretely: the system attaches, counts, drops, adapts its threshold live, and escalates selectively, each demonstrated with real evidence in Chapter Five.

The fourth objective, to evaluate the prototype against both attack and flash-crowd traffic, produced the central result of the whole dissertation - a static threshold that blocked twelve out of twelve legitimate sources, against an adaptive threshold that blocked none, with both configurations catching the attacker fully.

Taken together, and triangulating three independent strands of evidence - the functional tests confirming the mechanism works, the 0.86 accuracy of the classifier on a real benchmark dataset, and the flash-crowd comparison itself - the research question posed in Chapter One can be answered with a qualified yes: this design does maintain attack detection while measurably reducing flash-crowd false positives relative to a static baseline, at least under the conditions tested so far, with the honestly-stated limitations set out in Section 6.5.

7.2 Problems Encountered

Three problems shaped this project in ways worth recording, because each one changed a real decision rather than being a minor inconvenience.

The first was environmental rather than conceptual. The Tier-1 program, in its original form, would not compile on the development machine's particular kernel, failing with a compiler error buried inside an unrelated system header pulled in through the standard networking headers. Tracking this down took real time and involved reading compiler output far more carefully than expected, before the actual, small fix - switching to lighter headers and suppressing one warning - became obvious. Nothing about the program's logic changed; the fix was purely one of portability, but finding it was not obvious in advance.

The second was a measurement problem rather than an implementation one. An early attempt to read results back through the system's own dashboard produced numbers that made no sense, because the dashboard's own polling traffic was itself subject to the filtering being tested, and a low threshold silenced the very API being used to observe the system. Recognising this - that a defence mechanism can interfere with the tool measuring it - led straight to reading the kernel maps with bpftool instead, which removed the problem.

The third was a real limitation in the machine-learning tier, and one that is acknowledged rather than hidden: the features used at inference time, drawn from what the kernel maps can cheaply provide, are a simplified approximation of the richer flow-level features CIC-IDS-2017 itself contains. This does not affect the kernel filtering, the threshold adaptation, or the

flash-crowd result, but it does mean the classifier's real-world accuracy is likely somewhat lower than the 0.86 figure measured offline suggests, and closing this gap is the first item in the future work discussed below.

7.3 Self-reflection

7.3.1 Ideology About the Research Carried Out

Working through this project changed how the researcher thinks about what makes a DDoS defence good. Before starting, "good" meant catching attacks. By the end, it meant something narrower and, in this researcher's view, more honest: catching attacks without quietly punishing the people the system exists to protect in the first place. A defence that blocks everything technically has perfect detection and is also useless, and it was only by building and measuring the flash-crowd scenario first-hand that this stopped being an abstract point from the literature review and became something demonstrated with real numbers.

7.3.2 Benefits Gained

The clearest benefit was hands-on experience with eBPF and XDP, a technology this researcher had not touched before starting, and with reasoning carefully about the boundary between a fast, constrained kernel path and a slower, flexible userspace path. A second, less expected benefit was learning to train and honestly evaluate a machine-learning model against a recognised security benchmark, including learning to be suspicious of results that look too good, rather than simply reporting the highest accuracy figure obtained.

7.3.3 Learning Curves

The steepest part of the learning curve was the eBPF verifier itself and the surrounding toolchain, where the same code could behave differently on a different kernel version, and where problems generally had to be found by running the code rather than by reading documentation in advance. Debugging behaviour that spans the kernel and userspace boundary was demanding in a way that debugging a single Python script never is, since the cause of an odd symptom in one half was often sitting in the other. Undertaking a project of this technical depth with limited prior systems-programming experience was honestly challenging throughout, and the decision, explained in Chapter Three, to work in small, individually-tested

milestones rather than attempting the whole system at once is what made that challenge manageable rather than overwhelming.

7.4 Business Insight of Your Idea

7.4.1 Real-world Application Possibilities

The problem this dissertation addresses is not a niche one. Any service that experiences legitimate demand surges - a ticket release, a retail sale, breaking news traffic, a public-sector portal on a deadline day - faces exactly the trade-off studied here: a defence tuned aggressively enough to stop an attack can just as easily turn away the customers a surge represents, converting a moment of opportunity into a moment of lost business. A host-level, flash-crowd-aware filter of the kind built here could reduce both the damage caused by real attacks and the quieter, harder-to-notice cost of wrongly blocked legitimate users, using only standard Linux facilities rather than specialised or expensive hardware. That makes the underlying idea realistically adoptable by smaller organisations that could never afford enterprise-grade scrubbing services in the first place, which is precisely the gap identified as motivation in Section 1.5.

7.5 Future Recommendations

Several concrete extensions follow naturally from the limitations acknowledged above. Retraining the classifier on features that can actually be measured at inference time, rather than on the richer offline dataset features it currently learns from, would close the gap between the reported 0.86 accuracy and the classifier's real behaviour in production, and is the most important of these. Repeating the flash-crowd evaluation across truly separate physical machines, with the adaptive control loop choosing its own threshold during the run rather than a fixed stand-in value, and running each scenario several times to report a mean and a spread, would turn the current demonstration into a fuller statistical result. Extending the comparison to include at least one previously published method as a second baseline, rather than only a static threshold, would allow the system to be positioned quantitatively against the state of the art rather than only against a strawman. Beyond these, supporting IPv6 traffic and examining application-layer surges, both explicitly out of scope in Chapter One, would broaden the system considerably. Taken together, these steps mark a realistic path from a working, evidenced prototype towards a result solid enough to submit for peer review.

References

1. Cloudflare, "DDoS Threat Report for 2025 Q1," Cloudflare Blog, 2025. [Online]. Available: <https://blog.cloudflare.com/ddos-threat-report-for-2025-q1/>
2. Swif, "DDoS Attack Statistics for 2026: Record Sizes, Hyper-Volumetric Attacks, and the Botnets Behind Them," 2026. [Online]. Available: <https://www.swif.ai/blog/ddos-attack-statistics>
3. M. Abranches, O. Michel, E. Keller, and S. Schmid, "Efficient Network Monitoring Applications in the Kernel with eBPF and XDP," in Proc. 2021 IEEE Conf. Network Function Virtualization and Software Defined Networks (NFV-SDN), 2021.
4. A. Tolay, "eBPF-Based Real-Time DDoS Mitigation for IoT Edge Devices," arXiv:2508.00851, 2025.
5. N. Anand et al., "Enhancing intrusion detection against denial of service and distributed denial of service attacks: Leveraging extended Berkeley packet filter and machine learning algorithms," IET Communications, vol. 19, e12879, 2025.
6. T. Hara and M. Sasabe, "Practicality of in-kernel/user-space packet processing empowered by lightweight neural network and decision tree," Computer Networks, vol. 240, 2024.
7. M. Elzoghbi and H. He, "Kernel-Level LDoS Attack Detection in SDN Networks: An eBPF/XDP Framework with Dynamic Thresholding," Computer Networks, 2025.
8. Telecom Ramblings, "DDoS Attacks Grow 70% Larger in 2025 as Enterprises Move into the Crosshairs, Zayo Reports," 2026. [Online]. Available: <https://newswire.telecomramblings.com/2026/03/>
9. StationX, "DDoS Statistics 2026: Attack Data, Records and Trends," 2026. [Online]. Available: <https://app.stationx.net/articles/ddos-statistics>
10. "Efficient DDoS Detection and Mitigation in Cloud Data Centers Using eBPF and XDP," IEEE Conference Publication, 2025.
11. "Detection of Denial of Service Attack in Cloud Based Kubernetes Using eBPF,"

Applied Sciences (MDPI), vol. 13, no. 8, p. 4700, 2023.

12.Y. Zheng and J. Zhang, "In-Kernel CNN Inference for Edge Devices: An eBPF-Based Approach to Low-Latency and Resource-Efficient Processing," in Algorithms and Architectures for Parallel Processing, ICA3PP 2025, LNCS vol. 16387, Springer, 2026.

13.T. Farasat, J. Kim, and J. Posegga, "SmartX Intelligent Sec: A Security Framework Based on Machine Learning and eBPF/XDP," arXiv:2410.20244, 2024.

APPENDIX

Appendix A: Tier-1 Kernel Source Listing

The listing below is the complete source of `kernel/xdp_ddos.c`, the eBPF/XDP program that implements Tier 1 of the system, referenced throughout Chapters 4 and 5.

```
/*
 * xdp_ddos.c - Milestone M2+: Tier 1 DDoS filter at the XDP hook.
 *
 * For every IPv4 packet:
 *   1. Parse the Ethernet + IPv4 headers to get the source IP.
 *   2. Increment that IP's counter in the ip_count hash map.
 *   3. Compare against the live threshold in config map (key 0).
 *   4. XDP_DROP if over threshold; otherwise XDP_PASS.
 *
 * MAP CONTRACT (shared with the collector and the control loop)
 *   ip_count : BPF_HASH key=u32 (IPv4 src, network byte order),
value=u64
 *   config   : BPF_ARRAY key=u32 index 0 -> threshold (u64 packets)
 *   totals   : BPF_ARRAY key=u32 0=packets_seen 1=packets_dropped
2=packets_passed
 */

#include <uapi/linux/if_ether.h>
#include <uapi/linux/ip.h>

/* Per-source-IP packet counters. Max entries keeps map memory bounded.
```

```

*/
BPF_HASH(ip_count, u32, u64, 10240);

/* config[0] = drop threshold. Userspace writes a new value here for
live
    adaptive updates, without reloading this program. */
BPF_ARRAY(config, u64, 1);

/* Global tallies for the dashboard: seen / dropped / passed. */
BPF_ARRAY(totals, u64, 3);

static __always_inline void bump_total(u32 idx)
{
    u64 *v;
    u64 zero = 0;

    v = totals.lookup_or_init(&idx, &zero);
    if (v)
        __sync_fetch_and_add(v, 1);
}

int xdp_ddos(struct xdp_md *ctx)
{
    void *data_end = (void *) (long) ctx->data_end;
    void *data = (void *) (long) ctx->data;

    struct ethhdr *eth = data;
    /* Verifier requires every pointer arithmetic to be bounds-checked.
*/
    if ((void *) (eth + 1) > data_end)
        return XDP_PASS;

    /* Only IPv4; IPv6 / ARP / etc. pass through untouched. */
    if (eth->h_proto != bpf_htons(ETH_P_IP))
        return XDP_PASS;

    struct iphdr *iph = (void *) (eth + 1);
    if ((void *) (iph + 1) > data_end)
        return XDP_PASS;

```

```

    bump_total(0); /* packets_seen */

    u32 src = iph->saddr; /* already network byte order - use as map key
*/
    u64 zero = 0;
    u64 *cnt = ip_count.lookup_or_init(&src, &zero);
    if (!cnt)
        return XDP_PASS;

    __sync_fetch_and_add(cnt, 1);
    u64 count = *cnt;

    /* Default threshold if userspace has not written config[0] yet. */
    u32 cfg_key = 0;
    u64 *thr_ptr = config.lookup(&cfg_key);
    u64 threshold = thr_ptr ? *thr_ptr : 250;

    if (count >= threshold) {
        bump_total(1); /* packets_dropped */
        return XDP_DROP;
    }

    bump_total(2); /* packets_passed */
    return XDP_PASS;
}

```

Appendix B: Additional Test Cases

These cases extend Section 6.2 and include cases that surfaced real issues during development, which were then fixed - included here because they say as much about the system's robustness as the cases that simply passed.

ID	Test Case	Expected Result	Outcome
F-09	Clear counters through the dashboard while	ip_count and totals reset to zero; a clear event is recorded	Pass

ID	Test Case	Expected Result	Outcome
	the program is attached		
F-10	Toggle adaptive mode off, then back on	Threshold stops changing automatically while off; the control loop resumes computing and writing a new threshold once re-enabled	Pass
F-11	Attach the XDP program on a non-standard, newer kernel version	Initial attempt failed to compile against the full <code>linux/if_ether.h / linux/ip.h</code> headers on this kernel; switching to the lightweight <code>uapi</code> headers and adding a compiler flag resolved it without changing any detection logic	Pass (after fix)
F-12	Start the userspace stack twice against the same interface at once	Two independent programs attach and write to separate map instances, causing the dashboard to read an inconsistent view; identified as an operational hazard and documented as "run exactly one instance" rather than guarded in code, since guarding it was judged out of scope for a research prototype	Fail (documented limitation)
F-13	Run the demo attack generator without root privileges	Raw-socket creation is denied by the OS as expected; the generator falls back to a weaker ICMP-based flood so a demonstration can still proceed without <code>sudo</code>	Pass
F-1	Run the flash-crowd generator	Many short-lived connections are opened from distinct simulated ports; the	Pass

I D	Test Case	Expected Result	Outcome
4	against the attached interface	corresponding source counters in <code>ip_count</code> increase without triggering the drop threshold at a normal setting	
N - 0 3	Set an unusually low threshold while the dashboard itself is polling the API	The dashboard's own request traffic can itself cross a very low threshold and be dropped, silencing the dashboard; resolved for evaluation purposes by reading the BPF maps with <code>bpftool</code> instead, bypassing the API	Fail (documented limitation, evaluation methodology adjusted)
N - 0 4	Confirm dual persistence to SQLite and MongoDB Atlas	Every event and snapshot recorded by the collector is written to both the local SQLite database and the MongoDB Atlas mirror, so the system remains usable if the network connection to Atlas is unavailable	Pass

Cases F-12 and N-03 are included deliberately despite not being straightforward passes. Both were real issues found while developing and testing the system, and both shaped the design of the evaluation methodology described in Section 6.3 - in particular, the decision to measure results by reading the kernel maps straight rather than through the dashboard's own API.

Appendix C: Additional Use Case Specifications

The three specifications below expand on the use cases summarised in Section 4.4.1.

Use Case: View Live Status

Field	Description
Actor	Operator
Precondition	The backend service is running; the dashboard is loaded in a browser

Field	Description
Main flow	The dashboard polls the status, top-sources, and events endpoints once per second; the API reads the current state from the BPF maps (if attached) or from the database, and returns it as JSON; the dashboard updates the counters, chart, and event list
Postcondition	The operator sees an up-to-date view of traffic, drops, and recent events with no more than a one-second delay

Use Case: Set Threshold Manually

Field	Description
Actor	Operator
Precondition	The XDP program is attached
Main flow	The operator enters a new threshold value and submits it; the API validates the value, writes it into the config BPF map, and disables adaptive mode so the manual value is not immediately overwritten; the change is logged as an event
Postcondition	The kernel program reads the new threshold on the very next packet; no reload occurs

Use Case: Start Attack Traffic (Demonstration)

Field	Description
Actor	Traffic Generator (invoked by the Operator for demonstration purposes)
Precondition	A target address is configured; the generator script is available on the host
Main flow	The operator specifies a target and rate; the API launches the attack-generation process; generated traffic reaches the XDP hook and is counted and, once over threshold, dropped, exactly as any other traffic would be

Field	Description
Postcondition	The dashboard reflects the resulting rise in seen and dropped packets, and the attacking source appears in the top-sources list marked as dropped